

PHP pliki

Wykłady 12

Bartosz Lauks

17 maja 2026



- 1 Operacje na plikach
- 2 Ścieżki w PHP w kontekście operacji na plikach
- 3 Przesyłanie plików na serwer

- 1 Operacje na plikach
- 2 Ścieżki w PHP w kontekście operacji na plikach
- 3 Przesyłanie plików na serwer

Operacje na plikach

Wprowadzenie

- Umiejętność odczytu, zapisu i manipulacji plikami jest niezbędna w wielu zastosowaniach praktycznych, takich jak przechowywanie danych, generowanie raportów, zarządzanie konfiguracją aplikacji czy obsługa przesyłanych przez użytkowników plików.
- Pliki są podstawowym sposobem trwałego przechowywania danych w systemie operacyjnym. PHP, jako język skryptowy wykorzystywany do tworzenia dynamicznych aplikacji internetowych, oferuje szeroki zestaw funkcji do pracy z plikami i katalogami. Umożliwia to m.in.:
 - odczyt danych z plików,
 - zapis danych do plików,
 - tworzenie, usuwanie i modyfikowanie plików oraz katalogów,
 - sprawdzanie istnienia i właściwości plików,
 - operacje na strukturze katalogów.

Operacje na plikach

Podstawowe operacje na plikach

- Do podstawowego otwarcia oraz wypisania całej zawartości istniejącego pliku w PHP można użyć funkcji `readfile()`, która jako parametr przyjmuje ścieżkę do pliku.
- Co ciekawe sama funkcja w swoim działaniu zwraca zawartość pliku jako czysty tekst. Dodając `echo` dodatkowo otrzymamy długość tekstu w `int`

<https://localhost/lecture12/files/readfile.php>

Operacje na plikach

Otwieranie i zamykanie pliku

- PHP korzysta z funkcji `fopen()` do otwierania plików. Funkcja ta wymaga dwóch parametrów. Pierwszym jest ścieżka do pliku, a drugim tryb jego otwierania:
 - "r" - Odczyt - plik musi istnieć
 - "w" - Zapis - tworzy nowy plik lub nadpisuje istniejący
 - "a" - Dopisywanie - dane dopisywane są na końcu pliku
 - "r+" - Odczyt i zapis - plik musi istnieć
 - "w+" - Odczyt i zapis - nadpisuje plik lub tworzy nowy
 - "a+" - Odczyt i dopisywanie - dane dopisywane na końcu

Listing 1: Przykład fopen

```
1 $plik = fopen("plik.txt", "r"); // otwórz plik do odczytu
```

Operacje na plikach

Zamykanie pliku

- Po zakończeniu pracy z plikiem należy go zamknąć przy pomocy funkcji `fclose()`, gdzie jej parametrem musi być obiekt pliku otwartego przy pomocy `fopen()`.
- Warto o tym pamiętać z kilku powodów:
 - **Zwolnienie zasobów systemowych** - Otwieranie plików rezerwuje zasoby systemowe - takie jak uchwyty do plików.
 - **Zapis buforowanych danych** - Jeśli plik został otwarty do zapisu (np. z trybem "w", "a"), to dane mogą być buforowane w pamięci.
 - **Zabezpieczenie przed błędami** - niezamknięty plik może prowadzić do nieoczekiwanych błędów przy kolejnym otwarciu tego samego pliku lub spowodować, że inny proces nie będzie mógł go edytować.

<https://localhost/lecture12/files/fopen.php>

Operacje na plikach

Odczyt danych z pliku

- W PHP istnieje wiele funkcji, które służą do odczytu już plików otwartych przy pomocy `fopen()` np.:
 - `fgets()` - odczytuje plik linia po linii. Gdy cały plik zostanie odczytany, funkcja zwraca `false`.
 - `fread()` - kolejny sposób na odczytywanie zawartości pliku. W drugim parametrze można określić liczbę bajtów do odczytu.
 - `file_get_contents()` - nie wymaga wcześniejszego otwierania pliku za pomocą `fopen()`. Pobiera całą zawartość pliku jako łańcuch znaków (string).
Uwaga: nie jest to rozwiązanie optymalne dla dużych plików - Może powodować problemy z pamięcią przy bardzo dużych plikach (ładowanie całego pliku do RAM).

<https://localhost/lecture12/files/fopen2.php>

Operacje na plikach

Zapis danych do pliku

- PHP posiada wiele funkcji służących do zapisu danych w pliku - zarówno takich, które wymagają uprzedniego otwarcia pliku za pomocą **fopen()**, jak i takich, które tego nie wymagają.
 - **fwrite()** - służy do zapisu danych do wcześniej otwartego pliku. Jeśli chcemy zapisywać dane w nowej linii, należy dodać znak nowej linii (**\n**).
 - **file_put_contents()** - funkcja niewymagająca uprzedniego otwierania pliku. Służy do zapisywania danych do pliku na podstawie jego ścieżki. Domyślnie nadpisuje zawartość pliku; aby dopisać dane bez usuwania istniejącej zawartości, należy w trzecim parametrze dodać stałą **FILE_APPEND**.

<https://localhost/lecture12/files/fopen3.php>

Operacje na plikach

Sprawdzanie plików i katalogów

- Język PHP posiada szeroką gamę funkcji służących do sprawdzania stanu plików i katalogów. Do najistotniejszych należą:
 - `file_exists()` - sprawdza, czy plik istnieje. Jako parametr przyjmuje ścieżkę do pliku. Funkcja ta jest przydatna do zabezpieczenia programu przed błędami wynikającymi z operacji na nieistniejących plikach.
 - `is_file()`, `is_dir()` - zwracają wartość typu `bool` w zależności od tego, czy podana w parametrze ścieżka prowadzi odpowiednio do pliku lub katalogu.
 - `filesize()` - zwraca rozmiar pliku w bajtach.
 - `flemtime()` - zwraca datę ostatniej modyfikacji pliku w formacie czasu UNIX.

<https://localhost/lecture12/files/checkFileDir.php>

Operacje na plikach

Operacje na plikach i katalogach

- PHP posiada również funkcje umożliwiające pełne operowanie na systemie plików w systemie operacyjnym. Do najpopularniejszych należą:
 - **copy()** - służy do kopiowania pliku. Uwaga: funkcja obsługuje jedynie kopiowanie plików, nie katalogów.
 - **rename()** - umożliwia zmianę nazwy pliku lub przeniesienie pliku do innej lokalizacji.
 - **unlink()** - usuwa plik na podstawie ścieżki podanej w parametrze.
 - **mkdir()** - tworzy nowy katalog.
 - **rmdir()** - usuwa pusty katalog.

<https://localhost/lecture12/files/fileCopyRenameRemove.php>

- 1 Operacje na plikach
- 2 Ścieżki w PHP w kontekście operacji na plikach
- 3 Przesyłanie plików na serwer

Ścieżki w PHP w kontekście operacji na plikach

Ścieżki w PHP w kontekście operacji na plikach

- W PHP operacje na plikach (takie jak `fopen()`, `file_get_contents()`, `unlink()` itp.) wymagają podania ścieżki do pliku.
- Zrozumienie sposobu działania ścieżek jest kluczowe dla poprawnej i bezpiecznej pracy z systemem plików.
- Ścieżka do pliku może być:
 - **względna** (np. `data/plik.txt`)
 - **bezwzględna** (np. `/var/www/html/data/plik.txt`)

Ścieżki w PHP w kontekście operacji na plikach

Ścieżki względne

- Ścieżki względne są interpretowane względem bieżącego katalogu roboczego skryptu.
- Problemem ścieżek względnych jest ich zmienność w zależności od miejsca uruchomienia skryptu.
- Każdy program PHP uruchamiany w innej lokalizacji musi mieć odpowiednio zmodyfikowaną ścieżkę do pliku, na którym ma zostać wykonana.

Ścieżki w PHP w kontekście operacji na plikach

Ścieżki bezwzględne

- Ścieżki bezwzględne wskazują dokładne położenie pliku w systemie plików, niezależnie od lokalizacji bieżącego skryptu.
- Chroni to aplikację przed potencjalnymi błędami wynikającymi z niepoprawnie określonej ścieżki.
- Dodatkowo, stosowanie ścieżek bezwzględnych wpływa pozytywnie na przejrzystość kodu.
- Każda stała wskazująca ścieżkę do pliku odnosi się zawsze do tej samej lokalizacji, niezależnie od miejsca uruchomienia skryptu.

Ścieżki w PHP w kontekście operacji na plikach

Ścieżki bezwzględne cd.

- Jednak to rozwiązanie nie jest wolne od wad. Problemem może stać się system i serwer na którym hostujemy aplikacje. np.:
 - **apache2** (bezpośrednio w systemie Linux) -
`/var/www/html/projekt/dane/plik.txt`
 - **aplikacji XAMPP** - `/opt/lamp/htdocs/projekt/dane/plik.txt`
- Prowadzi to do tego samego problemu co w przypadku ścieżek względnych.
- Ewentualna zmiana serwera wymagać będzie od programisty wprowadzenie zmiany ścieżek w całej aplikacji.
- Aby tego uniknąć PHP udostępnia wiele stałych i funkcji do ustalania i diagnostyka bieżącej ścieżki.

<https://localhost/lecture12/files/path/>

Ścieżki w PHP w kontekście operacji na plikach

Ustalanie i diagnostyka bieżącej ścieżki

- `__DIR__` - ścieżka do katalogu bieżącego pliku
- `DIRECTORY_SEPARATOR` - stała w PHP zwracająca / lub \ w zależności od systemu operacyjnego.
- `getcwd()` - katalog roboczy (working directory)
- `basename()` - nazwa pliku ze ścieżki
- `dirname()` - katalog ze ścieżki
- `realpath()` - przekształca ścieżkę względną na bezwzględną

<https://localhost/lecture12/files/paths.php>

- 1 Operacje na plikach
- 2 Ścieżki w PHP w kontekście operacji na plikach
- 3 Przesyłanie plików na serwer

Przesyłanie plików na serwer

Formularz z przesyłaniem plików

- Przesyłanie plików to jedna z podstawowych funkcjonalności aplikacji internetowych.
- Dzięki niej użytkownicy mogą przysyłać na serwer obrazy, dokumenty czy inne pliki, które następnie mogą być przechowywane, przetwarzane lub udostępniane.
- W PHP przesyłanie plików odbywa się za pomocą formularzy HTML z odpowiednim ustawieniem atrybutów oraz specjalnych mechanizmów serwera do obsługi danych typu.

Przesyłanie plików na serwer

Konfiguracja php.ini

- Pierwszym krokiem implementacji funkcjonalności do przesyłania plików będzie modyfikacja pliku konfiguracyjnego **PHP**.
- Procedury tam zawarte mogą ograniczać możliwość przesyłania dużych plików.
- Aby umożliwić poprawne przesyłanie plików w aplikacji web w pliku konfiguracyjnym PHP [php.ini](#) należy ustawić:
 - **file_uploads = On** - umożliwia przesyłanie plików
 - **upload_max_filesize = 2M** - maksymalny rozmiar pojedynczego pliku
 - **post_max_size = 8M** - maksymalny rozmiar danych POST
 - **max_file_uploads = 20** - maksymalna liczba plików jednocześnie

Przesyłanie plików na serwer

Konfiguracja formularza HTML

- Formularz HTML nie umożliwia przesyłania plików, ponieważ domyślna wartość atrybutu **enctype**, który określa sposób kodowania danych formularza przesyłanych do serwera, na to nie pozwala.
- Bazowo ustawiony jest **enctype="application/x-www-form-urlencoded"**, co oznacza, że dane są przesyłane jako pary **klucz=wartość** w treści żądania (np. **name=Jan&email=jan@example.com**).
- Aby możliwe było przesyłanie plików, należy ustawić **enctype="multipart/form-data"**. Pozwala to na przesyłanie plików binarnych wraz z innymi danymi tekstowymi.

<https://localhost/lecture12/files/sendFile.php>

Przesyłanie plików na serwer

Jak PHP obsługuje pliki?

- W obecnej formie nasza aplikacji zwróci błąd ponieważ nie istnieje skrypt, który obsłuży żądania z przesyłanym plikiem.
- PHP udostępnia kilka możliwości na zapoznanie się informacjami o przesłanym pliku.
- Jednym z najprostszych sposobów jest użycie **superglobalnej** `$_FILES`, która przechowuje dane o wszystkich plikach przesłanych w bieżącym żądaniu.
- Jak i inne superglobalne jest to tablica asocjacyjna w której kluczem wartości jest zawartość atrybut `name=` znajdujący się w formularzu.

<https://localhost/lecture12/files/uploadServer.php>

Przesyłanie plików na serwer

Zapisywanie przesłanych plików w pamięci masowej

- Pliki przesłane za pośrednictwem bezstanowego protokołu HTTP istnieją tylko w czasie cyklu życia **request-response**.
- Każdy plik trafia do katalogu tymczasowego jeśli nie zostanie w żadnej formie zostanie usunięty z pamięci.
- Aby go zachować, należy go przenieść za pomocą funkcji `move_uploaded_file()`.

<https://localhost/lecture12/files/uploadServer.php>

Przesyłanie plików na serwer

Weryfikacja i bezpieczeństwo

- Weryfikacja plików powinna odbywać się po stronie klient i serwera.
- Czysty HTML udostępnia niewiele narzędzi do zabezpieczania warunków przesyłania plików.
- Ciekawym atrybutem jest `accept=`, które ogranicza możliwe rozszerzenia lub typy MIME.
- Bardziej zaawansowane sprawdzanie dodawanych plików do formularza wymagać będzie implementacji JS.

<https://localhost/lecture12/files/sendFile.php>

Przesyłanie plików na serwer

Weryfikacja i bezpieczeństwo po stronie serwera

- Zabezpieczenia po stronie klienta (przeglądarki) są nie wystarczające.
- Użytkownik w każdej chwili może zmodyfikować kod HTML i JS usuwając walidacje.
- Niezbędna jest walidacja po stronie serwera.
- Istnieją takie warunki walidacji, które można sprawdzić tylko po stronie serwera.

<https://localhost/lecture12/files/uploadServer.php>

Przesyłanie plików na serwer

Nadawanie unikalnej nazwy plikowi

- Obecnie zapisujemy na serwerze pliki z ich oryginalnymi nazwami.
- Ale co jeśli ktoś prześle nam plik z nazwą która jest już zajęta w pamięci masowej ?
- Nasza aplikacji można wygenerować błąd lub nowy plik nadpiszę stary. Powodując w ten sposób utratę danych.
- Aby się przed tym zabezpieczyć używa się algorytmów generujących unikalny, niepoważany ciąg znaków, którym zastępuje się oryginalną nazwą pliku.
- W PHP istnieją także funkcje `uniqid()`, która generuje unikalny identyfikator (**ID**) na podstawie bieżącego czasu z dokładnością do mikrosekund.
- W ten sposób będziemy mieć pewność, że nie powtórzy się taka sama nazwa pliku.

Przesyłanie plików na serwer

Przesyłanie wielu plików jednocześnie

- Czasami tworzona funkcjonalność może wymagać przesyłania wielu plików naraz.
- Obecnie formularz nie umożliwia wybrania kilku plików naraz, aby to umożliwić do pola **input** pliku należy dodać atrybut **multiple**.
- Dodatkowo musimy poinformować przeglądarkę oraz PHP że pole pliku będzie teraz zawierać tablicę danych, ponieważ może być wiele wartości o tej samej nazwie.
- Aby umożliwić przesyłanie tablicę danych należy w atrybucie name dodać `[]`.
- Niezbędne będą też zmiany w skrypcie po stronie serwera, aby przesyłanie kilku plików na raz.

<https://localhost/lecture12/files/sendFile.php>

Przesyłanie plików na serwer

Dobre praktyki dotyczących przesyłania plików w PHP

- **Zawsze weryfikuj pliki po stronie serwera** - Nie ufaj danym przesłanym przez użytkownika, nawet jeśli stosujesz walidację JavaScript.
- **Ogranicz rozmiar pliku** - Kontroluj to zarówno po stronie klienta (JavaScript), jak i w PHP.
- **Nadaj bezpieczną i unikalną nazwę pliku** - Nie używaj oryginalnej nazwy, może zawierać niebezpieczne znaki lub kolidować z istniejącym plikiem.
- **Ogranicz typy akceptowanych plików** - Nie pozwalaj użytkownikom przysyłać dowolnych plików (np. **.php**, **.exe**, **.bat**, **.js**).

Przesyłanie plików na serwer

Dobre praktyki dotyczących przesyłania plików w PHP

- **Przechowuj pliki poza katalogiem publicznym** - Jeśli to możliwe, przechowuj pliki poza **htdocs** i serwuj je np. przez skrypt PHP tylko po autoryzacji.
- **Nigdy nie pozwalaj użytkownikowi nadawać ścieżki docelowej** - Użytkownik nie może decydować, gdzie zapiszesz plik, może próbować np. zapisać plik jako **../../../config.php**
- **Chroń katalog **uploads/**** - Dodaj plik **.htaccess** (jeśli używasz Apache), aby np. zablokować wykonywanie skryptów

Dziękuję za uwagę !

Bartosz Lauks